Distributed Computing (CUDA)

Athanasios Agrafiotis
Computer Scientist
athaagrak@gmail.com

1. INTRODUCTION

The most important information is the difference between cpus and gpus and this is important because for a computer scientist should be capable to understand GPU hardware architectures(INVIDIA). To get the optimal performance it a gpu a scientint must be aware of the every unit, without a comprehensive knowledge of the hardware he cannot unlock the optimal performance. For better understanding we compare a cpu with a gpu, both cpus and gpus contains theoretical components or units. For example both have DRAM or global memory. There is a caching system in cpu and gpu but in different levels. The cpu contains three different levels of caching L1, L2 and L3, on similar the gpu one caching memory for each control system without to a specific level. Next we need to introduce the ALU(Arithmetic Large Unit), this unit is responsible for executing instructions in any application. The gpu has thousand of ALU and the cpu there is limited number of cpus. Given these similarities we can compare one ALU of the cpu and a thousand of small ALU that are called cores in the gpu. Comparing those the cpus ALUs are more consider as more productive. When refering to productive cpu ALU, we refer to speed, modern cpu achieve 3-4 GHz on the other hand a specific model of invidia A100 achieves to 765-1200 MHz(1.2 GHz). The ALU of cpu are designed for general tasks and executes a single thread only, on the other hand gpu are more specialized , designed to perform a specific tasks very efficiently and parallel so they execute many threads simultaneously (graphics). In general GPU the most of command are executed dependent, each instruction rellies on a predecessor. Each command is executed stores the result and pass the values to the next command. Meanwhile all these commands are run sequential. So the first command needs to execute to pass to the second command and at last to reach in the last command. For such scenario the cpu runs more productive however for the case that commands are indepedent the GPUs runs faster. If each command is independent runs parallel. So the absolute truth is sequential coding is running on cpu better.
In a gpu there is the actually the space that the hardware has attached a streaming multiproccesor(SM) alongside with SM there various levels of memories can observe such as L2 Cache Memory and bellow another SM the Device global memory, at the Scheduler and dispatcher. Each SM has a number of cores designed for a specific task some of these cores are dedicated for executing floating point operations while others execute integer operations along with the L1 cache memory with a scheduler unit , dispatcher unit and several registers. In the SM in terms of execution or functions the SM is quite diverse it can be found tensor cores designed for matrix multiplication operation, floating points to handling float operations and some special function units that are designed for some specific tasks executing the log operations. At last the SM features load and store units that are responsible for reading and writting values to the memory and this is just a quarter of SM. A real world example is the A-100 invidia with 108 SM each SM contains 64 multiprocessing cores this means that the entire gpu A-100 has around 7 thousand simple precision cores for floatings points and another 7 thousand cores for integer operations which is a very high count. Lately Meta et al (Konstantin and Sanders [3]) company announced it's ambition to construct the world most powerful supercomputer that relies on A-100 gpu.
In 1993, Jensen Huang founded the Nvidia Company. In just two 1995 NVIDIA launches it's first product, NV1. The device was quite simple in design featuring a gpu just 2 MB of bus with 64 MB of memory with one core or a single but the development has changed a model of invidia GeForce RTX 4090 Ti has close to 18176 cores compared with the first approach with memory of 24 Gigabytes in a matter of speed or frequency that is a crucial parameter affecting the application performance the base Clock is around 2355MHz as in the first approach was just 75MHz as 32x in the speed, that's a progress over the 25 last years of invidia. In 1997 RIVA 128 launches, that sells 1 million units sold in first four months NV3 and achieves 3D acceleration as plays a crucial role. In 1998 presents the GEFORCE with 4 cores and 32MB of memory.
The number of categories in invidia is the standard Normal Users and align with generations such as Tegra, Geforce, Quadro that are used for specifici reasons such as gaming and their model starts with (RTX). High performance computing HPC makes use of different architectures such as Tesla,Hopper,Ampere,Volta and their model starts with (H,A,V) that refers to cloud computing. A real world example is the GeForce RTX 3090 released in 2020 compared with A100 Tesla same year release. They both use Ampere

architecture but their targets are different, the RTX model is build for gaming in contrast the A-100 designed for HPC and cloud server. So when the invidia releases a new software or hardware architecture or both they announce a new architecture. The invidia has announced tegra for mobile devices, smart phones and tablets. The geforce target a regular consumers, Gaming, PC, HP graphics and video editing using RTX 3090. Next is Quadro target Professional Cards, Designers, Engineers, Graphic Design, Animation RTX 4 thousand series until RTX 6 thousand series and at last Tesla A100 that meta uses for high performance. To sum up we have GPU architecture at the top utilize to produce different generations and each generation has as target different market segments(Regular computers,HPC).

2. PROBLEM STATEMENT

Host denotes the CPU and its associated global memory or the RAM, while on the other side the device refers to the GPU and its global memory. This differentiation is crucial not just in terminology but for the way of implementing code. It influences code structure, data transfer, and memory handling. The most important is the relationship of hardware levels and the software parts. The structure of the GPU is hierarchical, it has four distinct levels of hardware organization. The top most level is the GPU itself, that encapsulates all units within this GPU. There are numerous streaming multiprocessors that each represent the second level of this hierarchy. Each multiprocessor is divided into several partitions making the third level of the hardware structure. A partition can be a number of 4 as you can observe onf figure closely on GPU photo. At the fourth level and most granular level, we find individual computational units within each partition. An example of computational units is the floating point core FP32, which is responsible for executing the floating point operations. These cores are the fundamental units for processing the computations within the GPU. According to this there is a similar structure to the software, so a cuda application needs to run on GPU.
How are the different segments of this will be allocated or assigned across the different hardware levels?
The answer is not simple, usually developers is to divide the GPU kernels from this application into smaller segments, called blocks or thread blocks. This segmentation or partitioning in the application mirrors the first two hierarchical levels of the the gpu.
The second level in the software involves different blocks being executed on the different SM.
So blocks considered as blocks and SM as workers, so who is responsible to assign these tasks to the workers?
Giga thread Unit is one of the most important units within the GPU. One of its key functions is to efficiently allocate these software blocks or task, or jobs across the various SM. It has connections to all SM and it is responsible for supplying them with the necessary block or task. So to continue a block is assigned by Gigathread to one SM that it has four partitions.
How should a single software piece called a block be distributed among the four hardware paritions that exist in SM?
if one block to one of these partitititions, this would lead to Underutilization, as it will waste the other three partitions. So in the next level of one partition of SM are the warp, if we deside to divide the first block here into four warps, then each warp can be allocated to a different partition within this SM. So warp and four partitions, in this case it is possible to assign one warp for each partition.if there are 32 warps per block, then each partition have four partitions and 32 warps. Then each partiion will execute eight warps, for more tasks it require a manager known as wrap scheduler. Because there is a possibility of many wraps in one partition, this is on the hardware side, but on the software side, a warp typically consists of a 32 threads. Each partition has core. Each thread within warp is going to be executed on a separate core within the partition. This setup ensures that every core in the partition is actively engaged in the computation process.

3. RELATED WORK

The research that explore the Volta architecture and its specifications as outlined in et al Z. Jia [2] scientifice research and the documentation. The volta architecture has enhancments in the V 100 compared to its predecessors. As it outlined starting form it's new streaming multiprocessor design to the second generation of the NVLink Link technology and so on. One of the standout aspectes in this architecture is the dedicated focus on the artificial intelligence and deep learning. This focues on the artificial intelligence and the deep learning is relevant when considering the introduction of the tensor core units for the first time in the Volta architecture. These tensor cores are a major breakthrough for the artificial intelligence, machine learning and deep learning computations. The tensor core units are engineered specifically to execute matrix operations, which are critical for the training and inference stages of the neural networks. The GV100 architecture in depth or hardware architecture that consists of around 21 billion transistors, which is considered a very large number however H100 contains 80 billion transistors, the quantity of transistors is indicative of the available computational resources. The main key features and enhancements in volta architecture that is a new design in the multiprocessor(SM). The new design of the SM in Volta architecture, is 50% more energy efficient than its predecessors, like the Pascal architecture. This enhancement is not just a technological , it is an economical one as well for businesses managing large scale computing systems like supercomputers. This means the V100 can potentially consume less energy than it's predecessors and offers a significant saving of computational power.
The second point is the improvements of the single and double precision cores. The streaming multiprocessor in volta architecture, it offers major improvements for the single precision and the double precision performance and happens with the same power amount. This translates into faster, more reliable computations, particularly for applications requiring high levels of precision, such as the scientific simulations. In a related scientific literature of et al Abdelkhalik [1] analyzes the instruction latencies within both volta and Pascal architectures, it also includes the turing architecture. It seems that high pecision numbers for instruction like the addition and substraction requires four cycles in contrast with the pascal that is close to six cycles. These illustrates the rapid advancement of instructions execution speed across these architectures in span of five to six years.
The tensor for the artificial intelligence computations, the tensor cores delivers twelve times faster TeraFlops/throughput.

TeraFlops is the number of instructions that get executed per second, so this unit delivers up to twelve speed for training models. The tensor cores are a different unit so another analysis in latencies is presented in the documentation compared to Ampere architecture. Important to note that the tensor cores perform operations on matrices, not just a single element. The parallel computation of float and integer operations are also introduced in volta architecture in contrast with the previous approaches like Fermi. This versatility is crucial for modern workloads that often involve a variety of data types and operations. An application that has a 50% of integer operations and another 50% of floating point operations. Instead of executing them sequentially , in volta are executed simultaneously. Advance in thread scheduling , combine the share memory and physical in one memory unit that increase the memory bandwidth as explain later in metrics. The software parts are also updated to be able to handle those operations such as the tensorflow. Each GPU consists of many SMs or many streaming Multiprocessors. Each SM consists of many units, including memories such as L1 and shared memory, as well as computatiotational units of executing different types of instructions like integer, single precision, double precision, (GPU,SM, execution units). Volta has each SM 64 hardware cores designed for single precision operations and equal number for processing the integer instructions, 32 cores for executing the double precision operations, and it also incorporates eight tensor cores. Volta is equipped in each SM with 84 cores multiplied by 64 single precision cores within the SM, this gives us the total number of cores dedicated to the single precision operations across the entire GPU. So the total number of cores for all data types, you might get around 20,000 computational units or 20,000 cores. Throughputs is the single precision operations and the Pascal has just 21, volta 31 and Ampere 78.NvLink is the is a way to connect gpu and cpu sharing processes.



Figure 1: Entire Volta GPU with 84 SM Units, global memory HBM2-memory controler,NVlink connected with other gpus,source:documentation Volta GPU

4. STREAMING MULTIPROCESSORS

Each gpu contains some streaming multiprocessors(SM), each SM is subdivided into several partitions, in which each is equipped with various computational units. There is a gpu , SM partitions and then the execution units or the computational units. There is a level zzero cache that stores the instructions for the GPU to execute. So this cache helps in speeding up the access to frequently used instructions (L0 instrcution cache). The instruction cache is not data cache because there is another level that is called L1 cache memory which is dedicated for the data caching accesses. Then is the warp scheduler is responsible for managing wraps. Each GPU application can be divided into multiple grids. Each grid can be divided into multiple blocks, and each block can be consisting of many threads. Each group of 32 threads withing a block is called a warp. These 32 threads withing the same warp execute the same instruction. And this is mentioned that the instructions within a warp using the single instruction mulitple thread fashion(signle instruction-multiple thread fashion).
for example if you want to add two vectors and store the result in a third one, is consider as a single instruction. But the addition of the example, on multiple elements on the first element and then the second element and so on. Each thread executes the same instruction which is the add operation but on a different element. So each warp is scheduler and executed in parallel all the 32 threads within the warp get executed on different cores, because within each partition there is 16 integer cores, special function units and tensor cores. So the 32 threads within a warp get executed on different cores simultaneously. The scheduler picks a strategy of how to execute the instruction and the dispatcher executes the strategy. And it also handles the whole warp the 32 threads within the warp per clock cycle. Then is the register file that contains a large number of registers, that stores and fast locations in the gpu. They hold the temporary data that the GPU's execution units need to perform operations on. And then there are the cores, the single precision, the integer, the tensor cores, the special functional. The whole sm contains four partitions and they all share the same L1 cache memory and the same texture memory.

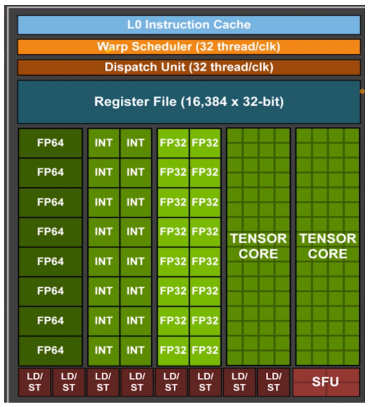


Figure 2

5. COMPUTE UNIFIED DEVICE ARCHITECTURE

Compute Unified Device Architecture is a parallel computing platform and application programming interface created by Invidia known as CUDA. CUDA allows software developers to use Invidia's graphic processing units for general purpose processing, an approach known as (GPGPU). CUDA is based on the C programming language, and this is an advantage as it allows developers write software that can execute on GPU. Cuda development is the Cuda compiler (nvcc).This compilrt is key to transforming high level CUDA code into a form that GPU understand. It can conver Cuda applications or Cuda codes int PTX codes, which are compatible with various

invidia architectures. Primarily, the nvcc compiler is utilized to transform CUDA code into an executable file that can be efficiently run on a GPU. Cude is enriched by a suite of optimized libraries, each one serving a specialized purpose. For example cuBLAS(Linear Algebra Subprograms) and cuFFT(Fast Fourier Transform) stand out of linear algebra and fast fourier transform operations. These librabies are optimized for boosting the performance significantly. Then there is the cuRAND(Random number generation), cuDNN(Deep Neural Network) these libraries are the backbone of efficient GPU programming, providing a pre-optimized ready to use that save time and enhance performance. The CUDA also provides the runtime APIs. Those APIs can manage the device initialization, the memory allocation, and the kernel launches. They are pivotal in managing how your application interacts with the GPU. So for example, for memory operations or data transfer between the CPU and the GPU, have used some of the following API functions. A function namely cudaMalloc() allocates memory on the GPU device, and it is used for allocating a specific amount of memory on the GPU. The other function is the cudaMemcpy() is used for copying data between the host or the CPU and the device, referred to the GPU. CUDA is also used to provide efficiency and stability. For debugging there is the NVIDIA Nsight systems for system performance analysis ,NVIDIA Nsight Compute for detailed CUDA profiling and analysis, CUDA-GBD for debuggin CUDA applications on Linux and Mac Os, CUDA-MEMCHECK tool to detect and diagnose memory errors in CUDA applications, all those tools is for refining and perfecting the GPU accelerated applications.

6. METRICS OF HARDWARE GPU

Each new architecture provide new advancement to software and hardware capabilities. These advancement provide high capabilities each across softwares. To evaluate the gpu performance is not a straightforward it requires to understand ten of units. So what are the most important parameters so we can compare a gpu among it's predecessors? For example when invidia increases the number of cores in it's architecture, is that enough so the performance can also increase or there are other factors?

The most important factors and metrics is the memory bandwith (memory speed + bus width). The memory bandwidth refers to how much data the memory can process per second and be GB/s. A real world example is a memory has a limit to process 32bits per second. let's also assoume that this gpu contains a small number of cores consider that each of the cores needs to read 32 bits of this memory, so the total are 128 bits, so it is possible to process all this requests in one second? and this is not possible it is require at least 4 seconds. In case of a memory of 128 bits/sec and we make the request of an architecture of 4 cores that reads 32 bit it can be done in one second. if the gpu is used for gaming it has as result a smoother gameplay. the bus width 2 bit is 2bits/sec 4 bit 4b/s. To sum up higher bus width better performance. 64 bit means 64 bits/s. The memory as a feature refers to how fast the memory can operate in gpu. The memory bandwidth is effected from the bud width, the memory speed and the memory technology (DDR4,DDR5,DDR6). A regular user gpu such as RTX3090 and a HPC gpu as A100 that share the same architecture Ampere. A100 bus width 5120 bit/s and RTX 384bit/s the difference is high but the memory speed is 1215Mhz in both gpus. A100 achieves 1.5GB/s and RTX model achieves 0.9GB/s.

Does the number of cores gives the advantage? if we compare two machines one with 100 cores and another with 200 cores. This metric is called TFLOPS(the core count + core speed). The total number of instructions per second that executed by the entire gpu. The higher number of cores allows more instructions to be executed simultaneously which results to higher throughput. An application with 200 instructions the in the first scenario it executed in two cycles compared the second that executes in one second. However if we compare base on one cycle time. The first will finish in one time and less time than the second. The cores cound and core speed are measured with throughput.

Another scenario one gpu with 100 cores and clock 100 MHz compared with 200 cores and clock 200 MHz. The second has higher cores and clock and this make the second the better choice as performance, but another key is the energy consumption. So the first choice can be also efficient.

When evaluating a new architecture is crucial to determine if any components have been edit, a new unit can enhance the gpu capabilities. In the Volta(2007) gpu has introduced the tensor cores that improve the gpu capabilities by far 8/1. It's fundamental to achieving the task efficiently and accurately. In the world of GPU computing and just like in CPUs, we have different ways to store numerical values depending on our needs. For example, an integer number. For integers, we can use eight, sixteen, thirty two, or even sixty four bits. The more bits you have, the larger number you can store. For example, an eight bit integer can store values between 0 and 255 if it is unsigned, or from -127 to 128 if the variable is declared as signed for a 64 bit integer, it is possible to much larger numbers up to like 18 quintillion. When it comes to numer with the decimal point, we use floating point representations. Float numbers are also come to different sizes for example, we have sixteen bits which is known as half precision. Then there is the thirty two bits and this is known as the single precision and finally the sixty four bits and this is known as double precision in C or CUDA programming languages, declaring a variable. If replace int with long, then this is often used as sixty four bits in the memory. Similarly on the other side, if we declare a variable with the float, then it uses thirdy bits int he memory, but if we replace float with double, then we use sixty precision bits in the memory. Half precision is a binary floating point format that occupies sixteen bits or two bytes in the computer memory (FP16). Half precision offers lower numerical accuracy but requires less computational resources, and this requires less computational resources, and this makes the half precision suitable for specific applications such as machine learning and graphic processing, where high precision may not be necessary. Single precision is also known as (FP32), and again thirty two here refers to the number of bits required to store this type of variable. This type offers a middle ground between the speed and the accuracy, and this is why the single precision, is the choice for a wide range of computing tasks. Then we have the double precision or the (FP64) type. This types is considered the precision powerhouse, and this because scientific computing and complex simulations wherever every decimal point can lead to a different result and rely on this type. This type ensures that simulations of phenomena like climate patterns or molecular structures are close to reality as possible. This approach and these different levels enable GPUs to be adaptable and utilized in a wide

range of industries and applications. That's the reason one of the research papers that investigated various GPU data types, it was discovered that double precision calculations require four execution cycles, by the single precision computations consume only two cycles, which indicates that the double precision is slower but provides higher accuracy.
In real world example just the summation of two numbers with default float (FP32) is two cycles. However, if we replace the float type with double the same operation, the addition here requires four cycles to complete. This change in cycle count is due to the increased precision and processing requirements of the double data type. The notated (π) in half precision is two decimal numbers for the single precision is sever decimal numbers and in double precision is fifteen decimal numbers. So it is clear that the half precision it would be insufficient for values that have a decimal values.

7. METRICS

The invidia suports an interface(smi) runs on command line. It offers monitoring and management capabilities for nvidia GPUs. Consider it as a powerful tool intefaces some key features and also monitor the GPU. The most critical areas that the interface covers is performance monitoring, Setting management and at last device information querying. The monitor includes the utilization of the GPU resources, the memory usage, the temperature and finally the power consumption. These data are essential for optimizing the resources of the GPU efficiently. The setting interface offers the flexibility to adjust or control the clock speeds of the GPU's memory and cores. Moreover, it can be used to determine some limits for the power consumption on the GPU. Finally querying device information this provides a comprehensive details about the GPU model, driver version. The most important aspects so far is the memory bandwidth and throughput. The occupancy of the GPU is a measure of the utilization of the resources in the GPU. In the realm of GPU performance, occupancy is categorized into two distinct types theoretical occupancy. Representing the idealized scenario, and achieved occupancy, indicating the actual usage of the GPUs, achieved occupancy the is the actual usage of the GPU's resources. The theoratical occupancy is calculated by dividing the number of warps utilized in a kernel by the maximum number of warps that can be accommodated per SM $\frac{warp\ used\ in\ a\ kernel}{max\ warps\ per\ SM}$.
In a real worl example, a cuda application designed to utilize 48 warps per SM. The 48 warps used in the application, matches the upper limit of warps per SM, so based on the upper limit calculating the ratio of these values provides a theoretical occupancy of 100%. In case the configuration of the application involves using for, 12 warps per SM, in this case, the ration 12 divided ny 48 indicates that the theoretical occupancy is reduced. Because the occupancy is related to another term called the allocated active blocks per SM, this term.
In vector addition application there is the input parameters in that kernel, i mean the number of threads per block and the total number of blocks. The occupacy refers to features such as the Block size and the block limit, that you can calculate the number of threads $block\ size\ *block\ limit$. So in an app it executes number of blocks based on the limit of a specific block size $block\ size\ *limited\ block\ size\ = total\ number\ of\ threads$ next to get the number of threads of each SM is divided by the theoretical Active Warps per SM and calculate the theoretical occupacy $\frac{total\ number\ of\ threads}{theoretical\ active\ warps\ per\ SM} = Occupancy$. The theoretical Occupancy it assumes optimal conditions is different than the achieved occupancy.
A GPU is made up of SM and that is one of these SM, each SM containing multiple paritions and in Ampere and Volta architectures. Each SM contains four partitions, in this hierarchy, applications run on the entire GPU. Blocks are designated to run on sms, and warps get executed on the partitions.
If an SM is uses 16 warps that conside the maximum number that can handle then the theoretical occupancy is 100%. But in a real world scenario the warps are not ready because they are awaiting a value from memory. During this wait, other warps may be scheduled for execution and at last for the scenario that warps encounter significant memory requests when the scheduler may encounter cycles where no warp is ready for execution (stall cycles). Each SM according to the figures of the paper has 4 partitions and each partition has 4 warps. The total number of warps utilized per SM is $number\ of\ warps\ per\ partition\ *\ number\ of\ warps\ per\ SM$ and the theoretical occupancy is $\frac{number\ of\ warps\ used\ in\ the\ application}{total\ number\ of\ warps} = theoretical\ occupancy.$
The warp scheduler starts talking to all warps and asking them which one of you is ready for executing one of its instructions. Given the instruction the scheduler will send it to cores that will use all of cores of wrap for one instruction (used even on different data). The cycles represent the time that it requires to complete one instruction but incase the computational core are occupied the scheduler is make use of LD(Load and Store Units) that requires at least 6 cycles to complete an instruction and these usually has as result stalled cycle if your values are sequentially.Hiding latenty means that giving a lot warps , a lot of threads, a lot of instructions to be executed on all the computational units. High occupancy does not always equate to high performance. Indicate that a significant portion of the GPU's performances are being utilized. Understand occupancy can help pinpoint performance issues. Low occupancy, on the other hand, suggests that there is a bottlenceck preventing the GPU from being fully utilized.
The four important features is Max Thread blocks/SM, more than 32 blocks per SM can run concurrently.
The second feature the max warps that can be executed simultaneously on a single SM. Warp limit controls how many active blocks can be allocated per SM.
The third feature is the max registers. If each thread requires 100 registers, then the total number of registers needed per SM would be 1024 * 100, which is equal to a hundred thousand, however the table limit is below this number. To run all max registers you need to decrease the numer of blocks as each block uses number of threads * block. However keep in mind that this number represents the maximum number of registers.
forth feature shared memory.All those feature is important to allocated active blocks per SM (AABS) that is the number of block runs concurrently on SM. As an example each feature sugests a different block limit, the block warp 4, the register 8 blocks and the shared memory limit 16 block however the software will choose 4 . So the SM in this case will have to select the minimum value among these constraints.

8. EXPERIMENT

Adding two vectors with around 1024 elements each, that the vector A contains 1023 elements and a vector B of 1023 and the result will be stored in a vector C. Element wise addition and each one will result to a C vector. The same application next compiles on CPU where the commands are sequential. An application of how cuda runs. A GPU kernel is running the 1024 instructions will be assigned to different cores to be executed in parallel. The number of instructions are two.
The first step in any CUDA application involves allocating space for you input and output data. This application needs to allocate memory for the three vectors, A, B and C. Next initialize vectors A and B values(CPU). The third step involves transferring these intialized values of A and B to the GPU, because our the addition operation needs to move to the device(GPU). Once the device has the data then it calculate the C vector and launch the GPU kernel. The copy the results back to the host(CPU). At the end to free the space that was used of the host(CPU) and device(GPU).

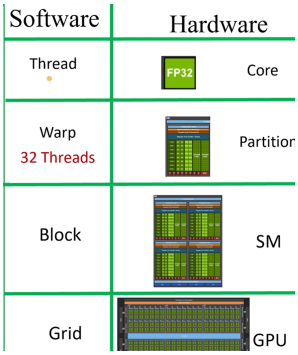
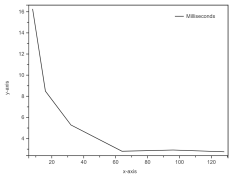


Figure 3


Figure 4: Results matrix addition 1024*1024*32, Grid Size 262144

9. RESULTS
The experiment with addition uses 1024 thread and 1 block. How does this example affect the allocated hardware resources, and also how is going to affect the performance of the application.
A thread operates on a GPU core, the core here is the smallest hardware unit in the GPU.
The warp executes in one out of 4(based on the architect of the gpu, that is located in SM) partitions.
The block runs on the entire SM,
and the grid executes on the entire GPU.
According to this and within the example application it uses one block and this indicates, 1 SM is used out of total 108 so the GPU utilization is close to 1%.
What actions can be takes to increase the number of SM allocated? using more SM, then you are using more resources which it increases the GPU utilization. In addition is not possible to increase threads and blocks. As each of the block need to use at least one partition that has at least 32 threads. So 1024/32 is 32 blocks that use at least one partition. Assigned 32 threads in partition of 4 will create 13 cycles of allocated space .
In the figure 4 you can observe the milliseconds that a gpu requires to make an addition between two matrices. Each matrix initialized in cpu and then passed to gpu to complete the equation with a specific number of blocks. The sm known as blocks does not play an important as the number of partitions is one until the 32 threads and the execution time is 15 for 8 threads and 10-6 for 32 threads. Next as the second partition is used the milliseconds are close to 2 for 2 to 3 partitions. When all the sm finaly used the occupancy is more than 73 ,the milliseconds achieve the highest decrease among the other values. For the calculation of grid size the equation that has been used is $\frac{vector\ size * threadBlock - 1}{threadBlock}$

10. CONCLUSION
10.1 Software
A performance analysis is concentrated on the number of cycles that a software can achieve under custom configuration. Some important metrics are the gpu time duration and the number of cycles that were active and they did not had any stall packages. When the number of cycles are high the correlated gpu will also slow down because is fully occupied. The other factor is the nuber of blocks and load store memory rate that have been used as it mention earlier it might create stall packages as it requires the doubled cycles in contrast to the time that one instruction requires to execute.
The memory bandwidth is like it has inverse relationship with the kernel time. The relationship describes how many warps are active and how does this have some relationship with the throughput.
How to calculate number of waves is the allocated active blocks , the grid size, and the number of SMs. Let's assume that the number of grid is 1048576 run 16 at the same time so is 1048576/16=65536/ 38 that are the number of SM that is equal to the number of waves that waves that is run to the application per SM, meaning 16 blocks per SM(even with stall packages), when a full cycle has ended , then the SM can start a new cycle.

10.2 Hardware
It it important for developers to be aware for the compute capability (CC). The compute capability of the Volta architecture is labeled as seven. This numerical designation reflects to specific set of features, the capabilities, the improvements presented in this architecture, which were cutting edge at the time of its release. Volta was notable for it's introduction of the tensor cores units, which significantly boosted the performance. In certain types of computations, particularly in the artificial intelligence and deep learning applications. Moving to the Ampere architecture, it's CC version is eight. This increment indicates substantial enhancement in both hardware and software compared to the Volta. The Ampere architecture has a various improvements, including more efficient and powerful tensor cores, larger memory bandwidth, and better energy efficiency, among other advancements. Different compute capability enable different features. This can include the types of mathemactical operations that the GPU can perform, the level of parallelism it can handle, the

memory access capabilities, and more other features.
For example when navigating to the link of the CUDA programming documentation, you would find many sections about the CUDA GPU programming. The compute capabilities allow developers to quickly find the available features that each version of gpu offers and if the gpu is compatible. One feature that is not supported in the 5.0 computer capabilities the half precision operations and tensor cores.
In addition the compute capability determines which version of CUDA toolkit it can run. To be more clear higi CC supports newer CUDA version. Another example PTX programming instructions can only perform on devices with compute capability above (CC:5.0).

11. REFERENCES

[1] H. Abdelkhalik, Y. Arafa, N. Santhi, and A.-H. Badawy. Demystifying the nvidia ampere architecture through microbenchmarking and instruction-level analysis, 2022.

[2] Z. Jia, M. Maggioni, B. Staiger, and D. P. Scarpazza. Dissecting the nvidia volta gpu architecture via microbenchmarking, 2018.

[3] K. Pilz, J. Sanders, R. Rahman, and L. Heim. Trends in ai supercomputers, 04 2025.